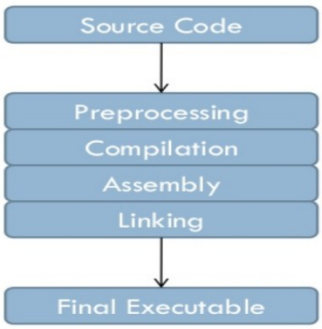
Experiment No :1

Date: 19-08-2025

Advanced GCC Commands In LINUX

GCC stands for GNU Compiler Collections. GCC Compiler is a very powerful and popular C compiler for various Linux distributions.

Compiling C files with gcc



GCC Compiler Options

1. **Specify the Output Executable Name**

   Source File Name is main.c

   In its most basic form, gcc compiler can be used as :

   ```
   gcc main.c
   ```

   The above command executes the complete compilation process and outputs an executable with name a.out.

2. **Writes the build output to an output file using –o option.**

   Use option -o, as shown below, to specify the output file name for the executable.

   Syntax:

   ```
   gcc main.c -o main
   ```

   The command above would produce an output file with name ‘main’.

3. **Produce only the compiled code using the -c option**

   To produce only the compiled code (without any linking), use the -c option.

Syntax:

```
gcc -c main.c
```

The command above would produce a file main.o that would contain machine level code or the compiled code.

4. Use compile time macros using -D option

The compiler option D can be used to define compile time macros in code.

Syntax:

```
$ gcc -Dname [options] [source files] [-o output file]
$ gcc -Dname=definition [options] [source files] [-o output file]
```

Example

```
// main.c
#include <stdio.h>
void main()
{
  #ifdef DEBUG
    printf("Debug run\n");
  #else
    printf("Release run\n");
  #endif
}
```

Build *main.c* and run it with DEBUG defined:

```
$ gcc -D DEBUG main.c -o main
$ ./main
```

Or build *main.c* and run it without DEBUG defined:

```
$ gcc main.c -o main
```

```
$ ./main
Release run
$
```

5. Link with shared libraries using -l option

The option -l can be used to link with shared libraries.

Syntax:

$ gcc [options] [source files] [object files] [-Ldir] -llibname [-o outfile]

Example:

$gcc -Wall main.c -o main –lCPPfile

The gcc command mentioned above links the code main.c with the shared library libCPPfile. So to produce the final executable 'main'.

6. Include directory of header files using –I option

The option –I can be used to include directory of header files

Syntax:

$ gcc -Idir [options] [source files] [object files] [-o output file]

Example:

proj/src/myheader.h:

```
// myheader.h
#define NUM1 5
//main.c
#include <stdio.h>
#include "myheader.h"
void main()
{
        int num = NUM1;
        printf("num=%d\n", num);
}
```

7. Debug information to be used by -g option

gcc -g generates debug information to be used by GDB debugger with 4 options –g0,-g1,-g,-g3

Syntax:

$ gcc -glevel [options] [source files] [object files] [-o output file]

$ gcc -g main.c -o main
$ gdb main

8. Setting the compiler optimization level using-O option

Set the compiler's optimization level several options such as -O0, -O1 ,-O2 ,-O3, -Os, -Ofast

Syntax:

$ gcc -Olevel [options] [source files] [object files] [-o output file]

$ gcc -O main.c -o main

9. Produce all the intermediate files using -save-temps function

Through this option, output at all the stages of compilation is stored in the current directory. This option produces the executable also.

Syntax:

$ gcc -save-temps source file

Example:

$ gcc -save-temps main.c

$ ls

a.out main.c main.i main.o main.s

main.i(preprocessed output) main.o(complied code) main.s(Assembly output)

GDB Commands in LINUX

The GNU Debugger, commonly abbreviated as GDB, is a command line tool that can be used to debug programs written in various programming languages such as C,C++,Ada, Fortran etc. The console can be using gdb command on terminal. gdb help us to find out watch or modify the variables in runtime, current state of the program, change the execute flow dynamically etc.

Compiling Programs with Debugging Information

To compile a C program with debugging information that can be read by the **GNU Debugger**, make sure the gcc compiler is run with the -g option.

Syntax:

$ gcc -g -o *output_file input_file*

Example:

$gcc -g -o main main.c

Common GDB Commands

1. **run**

   "run" will start the program running under gdb. The program that starts will be the one that you have previously selected with the file command, or on the unix command line when you started gdb. You can give command line arguments to your program on the gdb command line without saying the program name. i.e., it runs the loaded executable program with program arguments arg1 ... argn.

   Syntax:

   [r]un (arg1 arg2 ... argn):

   Example:

   (gdb) run 2048 24 4

2. **break**

A ``breakpoint'' is a spot in your program where you would like to temporarily stop execution in order to check the values of variables, or to try to find out where the program is crashing, etc. To set a breakpoint you use the break command. It set a breakpoint on either a function, a line given by a line number, or the instruction located at a particular address.

Syntax:

[b]reak <function name or file name: line# or *memory address>

Example:

(gdb) break main

Breakpoint 1 at 0x80488f6: file main.c, line 67.

3. **next**

"next" will continue until the next source line in the current function

Syntax:

[n]ext

Example:

(gdb) next

4. **print**

"print" *expression* will print out the value of the expression, which could be just a variable name.

Syntax:

[p]rint <expression>

Example:

(gdb) print list[0]@25

It prints out the first 25 values in an array called list

5. **Display**

Set an Automatic Display. It displays an expression every time the program stops.

Syntax:

display expression

Example:

(gdb) display/I $pc

It would display the next instruction after each step.

6. **help**

Gdb provides online documentation. Just typing help will give you a list of topics. Then you can type help *topic* /command to get information about that topic/command.

Syntax:

[h]elp [topic/command]

Example:

(gdb) help

gprof: LINUX GNU GCC Profiling Tool

Profiling allows you to learn where your program spent its time and which functions called which other functions while it was executing. Through profiling one can determine the parts in program code that are time consuming and need to be re-written. This helps make your program execution faster which is always desired.

GNU profiling tool 'gprof' produces an execution profile of C, Pascal, or Fortran77 programs. `Gprof` calculates the amount of time spent in each routine. Next, these times are propagated along the edges of the call graph. Cycles are discovered, and calls into a cycle are made to share the time of the cycle.

gprof is executing through the following steps:

- Have profiling enabled while compiling the code
- Execute the program code to produce the profiling data
- Run the gprof tool on the profiling data file (generated in the step above).

Step-1 : Profiling enabled while compilation

In this first step, we need to make sure that the profiling is enabled when the compilation of the code is done. This is made possible by adding the '-pg' option in

the compilation step. (-pg : Generate extra code to write profile information suitable for the analysis program gprof.)

```
$ gcc -pg testgprof.c new_testgprof.c -o testgprof
```

The option '-pg' can be used with the gcc command that compiles (-c option), gcc command that links(-o option on object files).

Step-2 : Execute the code

In the second step, the binary file produced as a result of step-1 is executed so that profiling information can be generated.

```
$ ./testgprof
```

If the profiling is enabled then on executing the program, file gmon.out (in built profile file for dynamic call graph and profile) will be generated.

```
$ ls
  gmon.out  testgprof  testgprof.c
```

Step-3 : Run the gprof tool

In this step, the gprof tool is run with the executable name and the above generated 'gmon.out' as argument. This produces an analysis file which contains all the desired profiling information.

```
$  gprof testgprof gmon.out > analysis.txt
```

Experiment No: 2

Date: 21-08-2025

MERGE TWO SORTED ARRAY AND STORE IN A THIRD ARRAY

AIM: To implement merge two sorted array.

ALGORITHM:

STEP 1: Start

STEP 2: Declare the size of the array

STEP 3: Declare the array.

STEP 4: Initialize the array.

STEP 5: While initializing the first array, copy the elements of the array to the merged array

STEP 6: While initializing the second array, copy the elements of the array to the merged array.

STEP 7: Now sort the merged array.

STEP 8: Display the resulting array.

STEP 9: Stop.

SOURCE CODE:

```
#include <stdio.h>
void main()
{
   int arr1[50],arr2[50],arr3[100],m,n,i,j,k=0;
   printf("\n Enter size of array 1: ");
   scanf("%d",&m);
   printf("\n Enter sorted elements of array 1: \n");
   for(i=0;i<m;i++)
     {
          scanf("%d",&arr1[i]);
     }
```

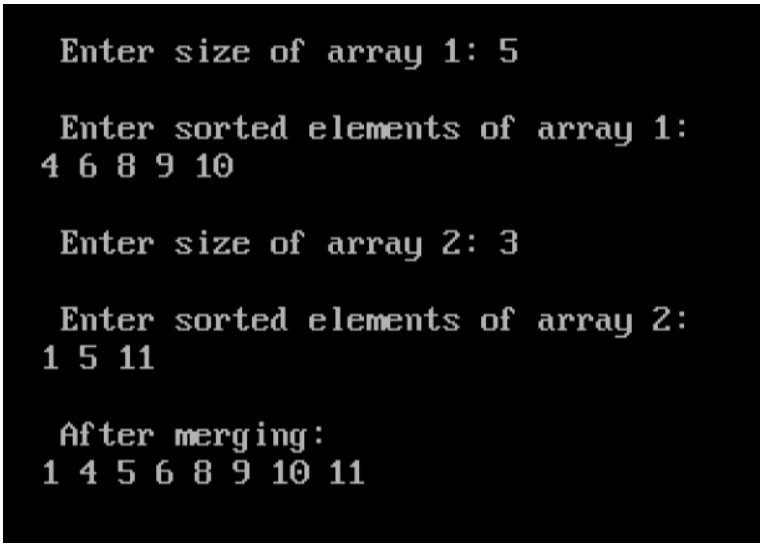
```
printf("\n Enter size of array 2: ");
scanf("%d",&n);
printf("\n Enter sorted elements of array 2: \n");
for(i=0;i<n;i++)
  {
        scanf("%d",&arr2[i]);
  }
i=0;
j=0;
while(i<m&&j<n)
  {
        if(arr1[i]<arr2[j])
              {
                    arr3[k]=arr1[i];
                    i++;
              }
        else
              {
                    arr3[k]=arr2[j];
                    j++;
              }
        k++;
  }
if(i>=m)
  {
        while(j<n)
        {
              arr3[k]=arr2[j];j++;
              k++;
        }
  }
```

```
    if(j>=n)
      {
            while(i<m)
                  {
                        arr3[k]=arr1[i];
                        i++;
                        k++;
                  }
      }
    printf("\n After merging: \n");
    for(i=0;i<m+n;i++)
      {
            printf("%d  ",arr3[i]);
      }

}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Enter size of array 1: 5
Enter sorted elements of array 1:
4 6 8 9 10
Enter size of array 2: 3
Enter sorted elements of array 2:
1 5 11
After merging:
1 4 5 6 8 9 10 11

Experiment No: 3

Date: 26-08-2025

CIRCULAR QUEUE USING ARRAY

AIM: To implement circular queue using array.

ALGORITHM:

STEP 1 : Include all the header files which are used in the program and define a constant 'MAX_SIZE'with specific value.

STEP 2 : Create a one dimensional array with above defined MAX_SIZE (queue[MAX_SIZE]). Define two integer variables 'front' and 'rear'. Initialize both with '-1'.

STEP 3 : Implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on circular queue.

ENQUEUE (ADD)

STEP1: Check if the queue is full.

If full, print "Queue is full- Overflow. Cannot enqueue." and return.

STEP2: If the queue is empty:

Set front and rear to 0.

STEP3: If the queue is not empty:

Update rear to (rear + 1) % MAX_SIZE to wrap around.

STEP4: Set queue[rear] to the given data.

Print "Data added to the queue."

DEQUEUE (DELETE)

STEP1: Check if the queue is empty.

If empty, print "Queue is empty. Cannot dequeue." and return.

STEP2: Print the data at queue[front] as it's removed.

STEP3: If front is equal to rear:

Set front and rear to -1 (indicating an empty queue).

STEP4: If the queue is not empty:

Update front to (front + 1) % MAX_SIZE to wrap around.

SEARCH

STEP1: Check if the queue is empty.

If empty, print "Queue is empty. Cannot search." and return -1.

STEP2: Set i to front.

STEP3: Repeat the following steps until i becomes (rear + 1) % MAX_SIZE:

a. If queue[i] is equal to the search key:

Print the key and its position in the queue.

Return i.

b. Update i to (i + 1) % MAX_SIZE to check the next element.

STEP4: If the loop completes without finding the key:

Print "Key not found in the queue."

Return -1.

DISPLAY

STEP1: Check if the queue is empty.

If empty, print "Queue is empty." and return.

STEP2: Print "Circular Queue: ".

STEP3: Set i to front.

STEP4: Repeat the following steps until i becomes (rear + 1) % MAX_SIZE:

a. Print queue[i] followed by a space.

b. Update i to (i + 1) % MAX_SIZE to move to the next element.

STEP5: End the display.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 5
int front = -1, rear = -1,i;
int queue[MAX_SIZE];
// Function to check if the queue is empty
int isEmpty()
        {
                return (front == -1 && rear == -1);
        }
// Function to check if the queue is full
int isFull()
        {
                return ((rear + 1) % MAX_SIZE == front);
        }
// Function to add an element to the circular queue
void enqueue(int data)
        {
                if (isFull())
                {
                        printf("Queue is full- Overflow. Cannot enqueue.\n");
                        return;
                }
             if (isEmpty())
                {
                        front = rear = 0;
                 }
               else
                {
                        rear = (rear + 1) % MAX_SIZE;
```

```
        }
    queue[rear] = data;
    printf("%d added to the queue.\n", data);
}

// Function to delete an element from the circular queue
void dequeue()
        {
                if (isEmpty()) {
                printf("Queue is empty-Underflow Cannot dequeue.\n");
                return;
        }
        printf("%d removed from the queue.\n", queue[front]);
        if (front == rear)
        {
                front = rear = -1;
        }
        else
        {
                front = (front + 1) % MAX_SIZE;
        }
        }

// Function to search for an element in the circular queue
int search(int key)
        {
                if (isEmpty())
                {
                        printf("Queue is empty. Cannot search.\n");
                        return -1;
                }
```

```
        i = front;
        do
                {
                        if (queue[i] == key)
                        {
                                printf("%d found at position %d in the queue.\n", key, i);
                                return i;
                        }
                i = (i + 1) % MAX_SIZE;
                } while (i != (rear + 1) % MAX_SIZE);
    printf("%d not found in the queue.\n", key);
    return -1;
}
// Function to display the circular queue
void display()
        {
                if (isEmpty())
                        {
                                printf("Queue is empty.\n");
                                return;
                        }
                printf("Circular Queue: ");
                i = front;
                do
                        {
                                printf("%d ", queue[i]);
                                i = (i + 1) % MAX_SIZE;
                        } while (i != (rear + 1) % MAX_SIZE);
                printf("\n");
        }
```

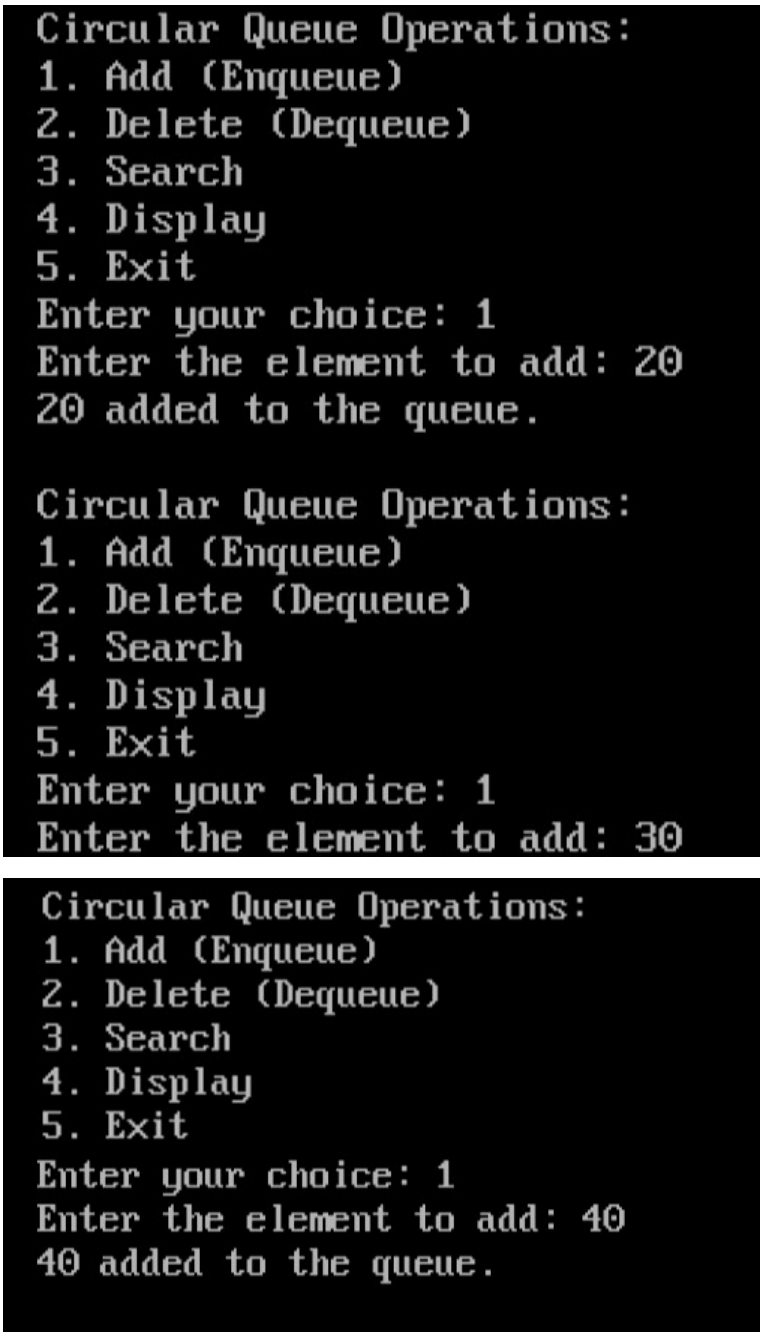
```
int main()
        {
                int choice, data, key;
        while (1) {
                printf("\nCircular Queue Operations:\n");
                printf("1. Add (Enqueue)\n");
                printf("2. Delete (Dequeue)\n");
                printf("3. Search\n");
                printf("4. Display\n");
                printf("5. Exit\n");
                printf("Enter your choice: ");
                scanf("%d", &choice);
                switch (choice) {
                        case 1:
                                printf("Enter the element to add: ");
                                scanf("%d", &data);
                                enqueue(data);
                                break;
                        case 2:
                                dequeue();
                                break;
                        case 3:
                                printf("Enter the element to search: ");
                                scanf("%d", &key);
                                search(key);
                                break;
                        case 4:
                                display();
                                break;
                        case 5:
                                exit(0);
```

```
                default:
                        printf("Invalid choice. Please try again.\n");
            }
        }

    return 0;
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 1
Enter the element to add: 20
20 added to the queue.
Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 1
Enter the element to add: 30
Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 1
Enter the element to add: 40
40 added to the queue.

```
Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 3
Enter the element to search: 30
30 found at position 2 in the queue.
```

```
Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 2
20 removed from the queue.
```

```
Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 4
Circular Queue: 30 40

Circular Queue Operations:
1. Add (Enqueue)
2. Delete (Dequeue)
3. Search
4. Display
5. Exit
Enter your choice: 5
```

Experiment No: 4

Date: 11-09-2025

a) SINGLY LINKED STACK

AIM: To implement singly linked stack

ALGORITHM:

STEP 1 : Include all the header files which are used in the program. And declare all the user defined functions.

STEP 2 : Define a 'Node' structure with two members data and next.

STEP 3 : Define a Node pointer 'top' and set it to NULL.

STEP 4 : Implement the main method by displaying Menu with list of operations and make suitable function calls in the main method.

PUSH()

STEP 1 : Create a newNode with given value.

STEP 2 : Check whether stack is Empty (top == NULL)

STEP 3 : If it is Empty, then set newNode → next = NULL.

STEP 4 : If it is Not Empty, then set newNode → next = top.

STEP 5 : Finally, set top = newNode.

POP()

STEP 1 : Check whether stack is Empty (top == NULL).

STEP 2 : If it is Empty, then display "Stack is Empty!!! Deletion is not possible!!!" and terminate the function

STEP 3 : If it is Not Empty, then define a Node pointer 'temp' and set it to 'top'.

STEP 4 : Then set 'top = top → next'.

STEP 5 : Finally, delete 'temp'. (free(temp)).

SEARCH()

STEP 1 : Read the element to be searched

STEP 2 : Iterate the loop until top != NULL

STEP 3 : If top -> data == srch_element then display element found message

STEP 4 : Else top = top -> link

DISPLAY()

STEP 1 : Check whether stack is Empty (top == NULL).

STEP 2 : If it is Empty, then display 'Stack is Empty!!!' and terminate the function.

STEP 3 : If it is Not Empty, then define a Node pointer 'temp' and initialize with top.

STEP 4 : Display 'temp → data --->' and move it to the next node. Repeat the same until temp reaches to the first node in the stack. (temp → next != NULL).

STEP 5 : Finally - Display 'temp → data ---> NULL'.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node* next;
};
struct Node* top = NULL;
int count = 0;
struct Node* temp;
void display()
{
   struct Node* temp = top;
   if (count == 0)
   {
        printf("\nStack is Empty!\n");
        return;
   }
   else
   {
```

```
        printf("\nThere are currently %d items:\n", count);
        while (temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}
void push(int el)
{
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    if (temp == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }
    temp->data = el;
    temp->next = top;
    top = temp;
    count++;
    display();
}
void pop()
{
```

```
    if (count == 0)
    {
        printf("\nStack is empty!\n");
        return;
    }
    temp = top;
    top = top->next;
    printf("\nItem to be removed is %d.\n", temp->data);
    free(temp);
    count--;
    display();
}
void search()
{
    int item, flag = 0,position=1;
    printf("Enter search element: ");
    scanf("%d", &item);

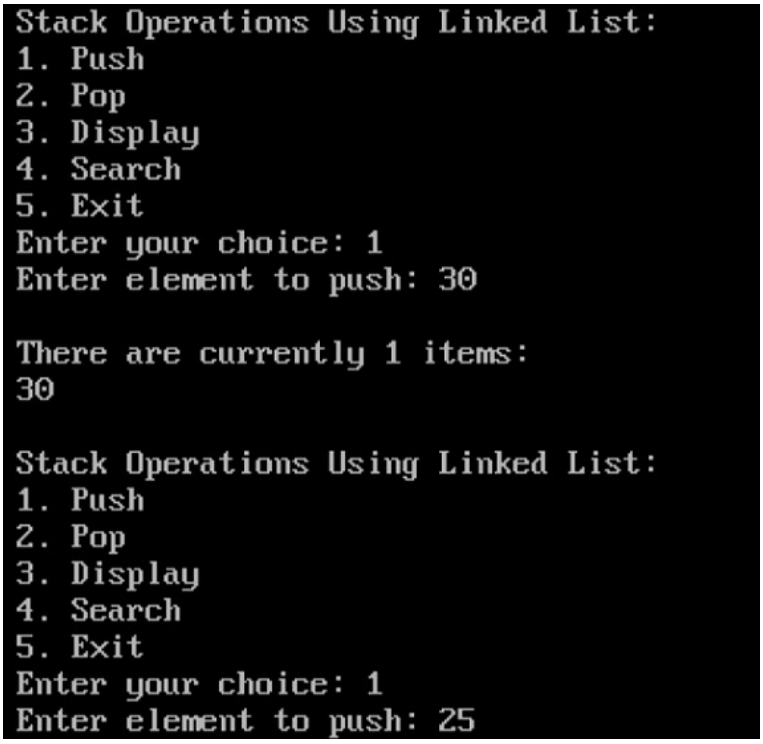
    if (top == NULL)
    {
        printf("Stack underflow\n");
        return;
    }
    temp = top;
    while (temp != NULL)
```

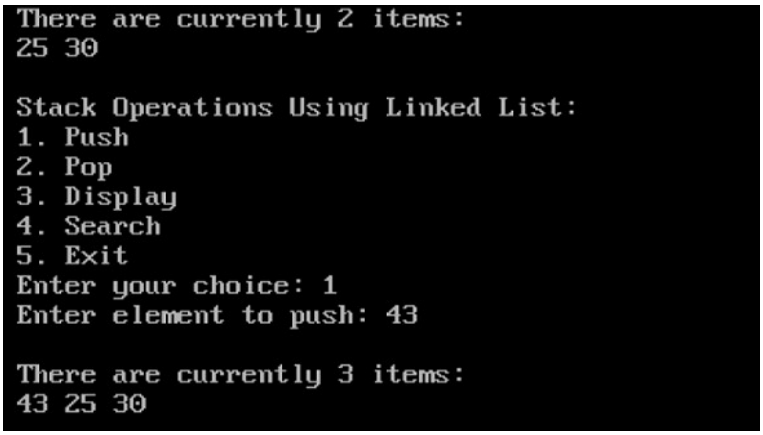
```
    {
        if (temp->data == item)
        {
            flag = 1;
            printf("\nElement %d found at position %d\n",item,position);
            break;
        }
        else
        {
            temp = temp->next;
            position++;
        }
    }
    if (flag == 0)
    {
        printf("\nElement %d not found\n", item);
    }
}
void main()
{
    int choice, element;
    while (1)
    {
        printf("\nStack Operations Using Linked List:\n");
        printf("1. Push\n2. Pop\n3. Display\n4. Search\n5. Exit\n");
```

```
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
           case 1:printf("Enter element to push: ");
                  scanf("%d", &element);
                  push(element);
                  break;
           case 2:pop();
                  break;
           case 3:display();
                  break;
           case 4:search();
                   break;
           case 5:exit(0);
                   break;
           default: printf("Invalid choice! Please try again.\n");
        }
}
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 1
Enter element to push: 30
There are currently 1 items:
30
Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 1
Enter element to push: 25


There are currently 2 items:
25 30
Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 1
Enter element to push: 43
There are currently 3 items:
43 25 30

```
Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 3

There are currently 3 items:
43 25 30

Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 4
Enter search element: 25

Element 25 found at position 2
```

```
Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 2

Item to be removed is 43.

There are currently 2 items:
25 30

Stack Operations Using Linked List:
1. Push
2. Pop
3. Display
4. Search
5. Exit
Enter your choice: 5
```

Experiment No: 4

Date: 18-09-2025

b) DOUBLY LINKED LIST

AIM: To implement doubly linked list

ALGORITHM:

INSERT:

Insert at the beginning of linked list

STEP 1 : Create a newNode with given value and newNode → previous as NULL.

STEP 2 : Check whether list is Empty (head == NULL)

STEP 3 : If it is Empty then, assign NULL to newNode → next and newNode to head.

STEP 4 : If it is not Empty then, assign head to newNode → next and newNode to head.

Insert at last of the linked list

STEP 1 : Create a newNode with given value and newNode → next as NULL.

STEP 2 : Check whether list is Empty (head == NULL)

STEP 3 : If it is Empty, then assign NULL to newNode → previous and newNode to head.

STEP 4 : If it is not Empty, then, define a node pointer temp and initialize with head.

STEP 5 : Keep moving the temp to its next node until it reaches to the last node in the list (until temp→next is equal to NULL).

STEP 6 : Assign newNode to temp→next and temp to newNode→previous.

Insert at specific position in the linked list

STEP 1 : Create a newNode with given value.

STEP 2 : Check whether list is Empty (head == NULL)

STEP 3 : If it is Empty then, assign NULL to both newNode → previous & newNode → next and set newNode to head.

STEP 4 : If it is not Empty then, define two node pointers temp1 & temp2 and Initialize temp1 with head.

STEP 5 : Keep moving the temp1 to its next node until it reaches to the node after which we want to insert the newNode (until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).

STEP 6 : Every time check whether temp1 is reached to the last node. If it is reached to the last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate tfunction. Otherwise move the temp1 to next node.

STEP 7 : Assign temp1 → next to temp2, newNode to temp1 → next, temp1 to newNode → previous, temp2 to newNode → next and newNode to temp2 → previous.

DELETE:

Deletion from beginning of the linked list

STEP 1: Check if the list is empty then the head is NULL, output "List is empty. Cannot delete from an empty list." and exit.

STEP 2 : If it is not empty, create a temporary pointer temp and set it to the current head node.

STEP 3 : If temp → next is NULL (i.e., there is only one node in the list), thenFree the memory allocated for temp, head->NULL and Exit.

STEP 4 : If there is more than one node in the list, then update head to point to temp → next , set the previous pointer of the new head to NULL and free the memory allocated for temp.

STEP 5 : Exit

Deletion from end of the list

STEP 1 : Check whether list is Empty (head == NULL)

STEP 2 : If it is Empty, then display 'List is Empty' and terminate the function.

STEP 3 : If it is not Empty then, define a Node pointer 'temp' and initialize with head.

STEP 4 : Check whether list has only one Node (temp → previous and temp → next both are NULL)

STEP 5 : If it is TRUE, then assign NULL to head and delete temp. And terminate from the function.(Setting Empty list condition)

STEP 6 : If it is FALSE, then keep moving temp until it reaches to the last node in the list.(until temp→ next is equal to NULL)

STEP 7 : Assign NULL to temp → previous → next and delete temp.

Deleting a specific node from the linked list

STEP 1 : Check whether list is Empty (head == NULL)

STEP 2 : If it is Empty then, display 'List is Empty' and terminate the function.

STEP 3 : If it is not Empty, then define a Node pointer 'temp' and initialize with head.

STEP 4 : Keep moving the temp until it reaches to the exact node to be deleted or to the last node.

STEP 5 : If it is reached to the last node, then display 'No such element found' and terminate the function.

STEP 6 : If it is reached to the exact node which we want to delete, then check whether list is having only one node or not

STEP 7 : If list has only one node and that is the node which is to be deleted then set head to NULL and delete temp (free(temp)).

STEP 8 : If list contains multiple nodes, then check whether temp is the first node in the list (temp = = head).

STEP 9 : If temp is the first node, then move the head to the next node (head = head → next), set head of previous to NULL (head → previous = NULL) and delete temp.

STEP 10 : If temp is not the first node, then check whether it is the last node in the list (temp → next = = NULL).

STEP 11 : If temp is the last node then set temp of previous of next to NULL (temp → previous→ next = NULL) and delete temp (free(temp)).

STEP 12 : If temp is not the first node and not the last node, then set temp of previous of next to temp of next (temp → previous → next = temp → next), temp of next of previous to temp of previous (temp → next → previous = temp → previous) and delete temp(free(temp)).

DISPLAY:

STEP 1 : Check whether list is Empty (head == NULL)

STEP 2 : If it is Empty, then display 'List is Empty' and terminate the function.

STEP 3 : If it is not Empty, then define a Node pointer 'temp' and initialize with head.

STEP 4 : Display 'NULL <--- '.

STEP 5 : Keep displaying temp → data with an arrow (<===>) until temp reaches to the last node

STEP 6 : Finally, display temp → data with arrow pointing to NULL (temp → data ---> NULL).

SOURCE CODE:

```
#include<stdio.h>

#include<stdlib.h>

void insbeg();

void insmid();

void insend();

void delbeg();

void delmid();

void delend();

void display();

void creation();

void searching();

struct node

{

        int data;

        struct node *next,*prev;

};

struct node *head,*newnode,*temp,*temp1;
```

```
void main()
{
	int choice=0;

	do
	{
		printf("Enter the choice \n1.Creation \n2.Insetion at Beginning \n3.Insertion at
			Middle \n4.Insertion at End \n5.Deletion from Beginning  \n 6.Deletion
			from Middle \n7.Deletion from End \n8.Searching \n9.Exit \nPress a
			number in the range :");
		scanf("%d",&choice);
		switch(choice)
		{
			case 1:creation();display();break;
			case 2:insbeg();display();break;
			case 3:insmid();display();break;
			case 4:insend();display();break;
			case 5:delbeg();display();break;
			case 6:delmid();display();break;
			case 7:delend();display();break;
			case 8:searching();display();break;
			case 9:exit(0);break;
```

```
                }

        }while(choice!=9);

}
void creation()
{
        int ch=1;
        head=0;
        while(ch)
        {
                newnode=(struct node*)malloc(sizeof(struct node));
                printf("Enter the element to be inserted\t:");
                scanf("%d",&newnode->data);
                newnode->next=NULL;
                newnode->prev=NULL;
                if(head==0)
                {
                        head=temp=newnode;
                }
                else
```

```
                {
                        temp->next=newnode;
                        newnode->prev=temp;
                        temp=newnode;
                }
                printf("Do you want to continue(1/0)");
                scanf("%d",&ch);
        }
}
void display()
{
        int count=0;
        temp=head;
        printf("\n The Linked List is:\n");
        printf("START->");
        while(temp!=NULL)
        {
                count=count+1;
                printf("%d->",temp->data);
                temp=temp->next;
        }
```

```
        printf("END");
        printf("\n The number of nodes is:\t %d\n\n",count);
}
void insbeg()
{
        newnode=(struct node*)malloc(sizeof(struct node));
        printf("Enter the element to be inserted\t:");
        scanf("%d",&newnode->data);
        newnode->next=head;
        newnode->prev=NULL;
        head=newnode;
}
void insmid()
{
        int pos,count=0,i=1;
        temp=head;
        while(temp!=NULL)
        {
                count=count+1;
                temp=temp->next;
        }
```

```
printf("Enter the position after which the element to be inserted\t:");
scanf("%d",&pos);
if(pos>count)
{
        printf("Invalid position");
}
else
{
        temp=temp1=head;
        while(i<pos)
        {

                temp1=temp;
                temp=temp->next;
                i++;
        }
        printf("Enter the element to be inserted\t:");
        newnode=(struct node*)malloc(sizeof(struct node));
        scanf("%d",&newnode->data);
        newnode->next=temp,
        temp->prev=newnode;
```

```
                newnode->prev=temp1;
                temp1->next=newnode;
        }
}
void insend()
{
        newnode=(struct node*)malloc(sizeof(struct  node));
        printf("Enter the element to be inserted\t:");
        scanf("%d",&newnode->data);
        newnode->next=NULL;
        temp=head;
        while(temp->next!=NULL)
        {
                temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
}
void delbeg()
{
        if(head==NULL)
```

```
        printf("Node absent\n");
        temp=head;
        head=temp->next;
        head->prev=NULL;
        free(temp);
}
void delmid()
{
        int del;
        if(head==NULL)
        printf("Node absent\n");
        printf("Enter the data to delete\t:");
        scanf("%d",&del);
        temp=head;
        while(temp->data!=del)
        {
                if(temp->next==NULL)
                {
                        printf("Given node is not found \n");
                        break;
                }
```

```
            else
            {
                temp=temp->next;
            }
        }
        while(temp->data==del)
        {
        if(temp==head)
        {
            head=NULL;
            free(temp);
        }
        else
        {
            temp->prev->next=temp->next;
            free(temp);
        }
        }
}
void delend()
{
```

```
        if(head==NULL)
        printf("Node absent\n");
        temp=head;
        while(temp->next!=NULL)
        {
                temp1=temp;
                temp=temp->next;
        }
        temp1->next=NULL;
}
void searching()
{
        int item,i=1,flag=0;
        printf("Enter the element to search:");
        scanf("%d",&item);
        temp=head;
        while(temp!=NULL)
         {
                 if(temp->data==item)
                         {
```

```
                printf("\n\nITEM FOUND AT %d POSITION\n\n",i);
                flag=flag+1;
            }
            i=i+1;
            temp=temp->next;
        }
        if(flag==0)
        {
            printf("\nSORRY.. ITEM NOT FOUND !! \n\n");
        }
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:

```
Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
Press a number in the range :1
Enter the element to be inserted :10
Do you want to continue(1/0): 1
Enter the element to be inserted :20
Do you want to continue(1/0): 1
Enter the element to be inserted :30
Do you want to continue(1/0): 0
```

```
 The Linked List is:
START->10->20->30->END
 The number of nodes is:  3

Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
Press a number in the range :2
Enter the element to be inserted :5

 The Linked List is:
START->5->10->20->30->END
 The number of nodes is:  4
```

```
Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
```

```
Press a number in the range :3
Enter the position after which the element to be inserted:2
Enter the element to be inserted:15

 The Linked List is:
START->5->15->10->20->30->END
 The number of nodes is:  5

Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
Press a number in the range :4
```

```
Enter the element to be inserted: 50

 The Linked List is:
START->5->15->10->20->30->50->END
 The number of nodes is: 6

Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
```

```
Press a number in the range :8
Enter the element to search:30


   ITEM FOUND AT POSITION 5


 The Linked List is:
START->5->15->10->20->30->50->END
 The number of nodes is: 6

Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
```

```
 Press a number in the range :5

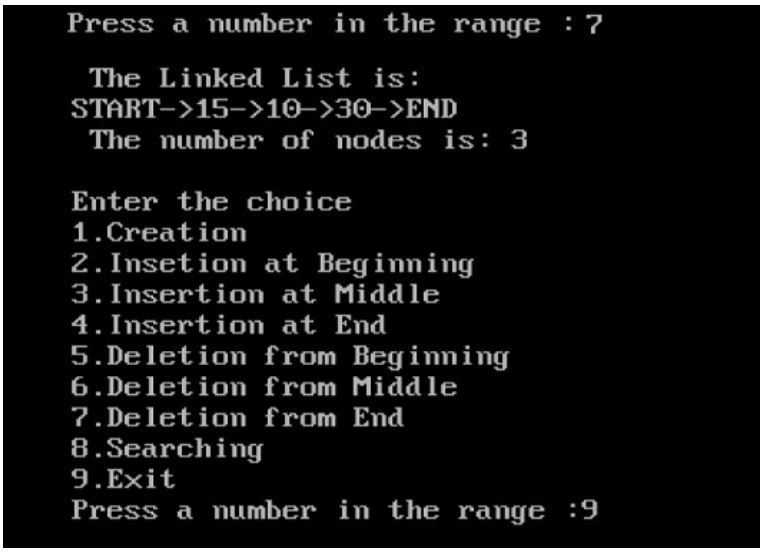
  The Linked List is:
START->15->10->20->30->50->END
 The number of nodes is: 5

 Enter the choice
 1.Creation
 2.Insetion at Beginning
 3.Insertion at Middle
 4.Insertion at End
 5.Deletion from Beginning
 6.Deletion from Middle
 7.Deletion from End
 8.Searching
 9.Exit
```

```
Press a number in the range :6
Enter the data to delete: 20

 The Linked List is:
START->15->10->30->50->END
 The number of nodes is: 4

Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
```


Press a number in the range : 7
The Linked List is:
START->15->10->30->END
The number of nodes is: 3
Enter the choice
1.Creation
2.Insetion at Beginning
3.Insertion at Middle
4.Insertion at End
5.Deletion from Beginning
6.Deletion from Middle
7.Deletion from End
8.Searching
9.Exit
Press a number in the range :9

Experiment No: 5

Date: 25-09-2025

SET OPERATIONS USING BIT STRING

AIM: To perform Union, Intersection and Difference operations using Bitstring.

ALGORITHM:

input()

STEP 1 : Read size and elements of Set 1 and Set 2

union()

STEP 1 : for i 1 to 9 do

STEP 1.1 : if(a[i] != b[i]) then set c[i]=1

STEP 1.2 : else set c[i] = a[i]

STEP 2 : End for

STEP 3 : Display c

INTERSECTION()

STEP 1 : for i 1 to 9 do

STEP 1.1 : if(a[i] == b[i])then set c[i]=a[i]

STEP 1.2 : else set c[i] = 0

STEP 2: End for

STEP 3 : Display c

difference()

STEP 1 : for i 1 to 9 do

STEP 1.1 : if(a[i] == 1 && b[i] == 0) then set c[i]=1

STEP 1.2: else set c[i] = 0

STEP 2 : End for

STEP 3 : Display c

SOURCE CODE:

```
//Universal Set is{1,2,3,4,5,6,7,8,9}

#include<stdio.h>

#include<conio.h>

void input();
```

```
void output();
void setunion();
void intersection();
void difference();
int a[]={0,0,0,0,0,0,0,0,0,0},
b[]= {0,0,0,0,0,0,0,0,0,0},
c[]={0,0,0,0,0,0,0,0,0,0};
void main()
{
        int ch,wish;

        do
        {
                printf("\n MENU \n");
                printf("\n 1.Input \n 2.Union \n 3.Intersection \n 4.Difference \n");
                printf("\n Enter your choice:");
                scanf("%d",&ch);
                switch(ch)
                {
                        case 1: input();
                                break;
                        case 2: setunion();
                                break;
                        case 3: intersection();
                                break;
```

```
                case 4: difference();
                        break;
                 default:printf("Wrong Choice");
            }
            printf("\nDo you wish to continue?(1/0) \n");
            scanf("%d",&wish);
        }while(wish==1);
}
void input()
{
        int n,ele,i;
        printf("Enter the size of first set:");
        scanf("%d",&n);
        printf("\n Enter the elements:\n");
        for(i=1;i<=n;i++)
        {
            scanf("%d",&ele);
            a[ele]=1;
        }
        printf("Enter the size of second set:");
        scanf("%d",&n);
        printf("\n Enter the elements:\n");
        for(i=1;i<=n;i++)
        {
            scanf("%d",&ele);
```

```
            b[ele]=1;
        }
        printf("\n Set 1: { ");
        for(i=1;i<=9;i++)
        {
            printf("%d",a[i]);
        }
        printf(" }");
        printf("\n Set 2: { ");
        for(i=1;i<=9;i++)
        {
            printf("%d",b[i]);
        }
        printf(" }");
}
void output(int c[])
{
        int i;
        printf("\n Set : { ");
        for(i=1;i<=9;i++)
        {
            if(c[i]!=0)
                    printf("%d \t",i);

        }
```

```
        printf(“ }”);
}
void setunion()
{
        int i,c[10];
        for(i=1;i<=9;i++)
        {
                if(a[i]!=b[i])
                                c[i]=1;

                else
                        c[i]=a[i];
        }
        printf(“Union of  Set 1 and Set 2 is: { “);
        for(i=1;i<=9;i++)
        {
                printf("%d",c[i]);
        }
        printf(“ }”);
        output(c);
}
void intersection()
{
        int i,c[10];
        for(i=1;i<=9;i++)
```

```
        {
        if(a[i]==b[i])
                        c[i]=a[i];
        else
                c[i]=0;
        }
        printf("Intersection of  Set 1 and Set 2 is: { ");
        for(i=1;i<=9;i++)
        {
                printf("%d",c[i]);
        }
        printf(" }");
        output(c);
}
void difference()
{
        int i,c[10];
        for(i=1;i<=9;i++)
        {
        if(a[i]==1&&b[i]==0)
                                c[i]=1;
        else
                c[i]=0;
        }
        printf("Difference of Set 1 and Set 2 is: { ");
```

```
        for(i=1;i<=9;i++)
        {
                printf("%d",c[i]);
        }
        printf(" }");
        output(c);
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:

```
  MENU

  1.Input
  2.Union
  3.Intersection
  4.Difference

  Enter your choice:1
Enter the size of first set:5

  Enter the elements:
1 2 3 4 5
Enter the size of second set:5

  Enter the elements:
6 7 8 9 3

  Set1: {1 1 1 1 1 0 0 0 0 }
  Set2: {0 0 1 0 0 1 1 1 1 }
Do you wish to continue?(1/0): 1
```

```
 MENU

 1.Input
 2.Union
 3.Intersection
 4.Difference

 Enter your choice:2
Union of Set1 and Set2 is : { 1 1 1 1 1 1 1 1 1 }
 Set :{ 1 2 3 4 5 6 7 8 9 }
Do you wish to continue?(1/0): 1

 MENU

 1.Input
 2.Union
 3.Intersection
 4.Difference

 Enter your choice:3
Intersection of Set1 and Set2 is : { 0 0 1 0 0 0 0 0 0 }
 Set :{ 3 }
Do you wish to continue?(1/0): 1
```

```
 MENU

 1.Input
 2.Union
 3.Intersection
 4.Difference

 Enter your choice:3
Intersection of Set1 and Set2 is : { 0 0 1 0 0 0 0 0 0 }
 Set :{ 3 }
Do you wish to continue?(1/0): 1

 MENU

 1.Input
 2.Union
 3.Intersection
 4.Difference

 Enter your choice:4
Difference of Set1 and Set2 is : { 1 1 0 1 1 0 0 0 0 }
 Set :{ 1 2 4 5 }
Do you wish to continue?(1/0): 0
```

Experiment No: 6

Date: 09-10-2025

DISJOINT SET OPERATIONS

AIM: To perform Create , Union , Find operations of Disjoint Set.

ALGORITHM:

makeSet()

STEP 1 : For I to dis.n

STEP 2 : parent[x]=x

STEP 3 : rank[x]=0

STEP 4 : end for

Union()

STEP 1 : LINK(findSet(x), FindSet(y)

STEP 2 : LINK(x,y)

STEP 2.1 : if rank[x]>rank[y] then p[y]=x

STEP 2.2 : else p[x]=y

STEP 2.3 : if rank[x] == rank[y]

STEP 2.4 : then rank[y]==rank[y]+1

findSet(x)

STEP 1 : if x == p[x] then p[x] = findSet(p[x])

STEP 2 : return p[x]

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct DisJoint{
        int parent[10];
        int rank[10];
        int n;

}dis;
```

```
void makeset()
{
	int i;
	for(i=0;i<dis.n;i++){
		dis.parent[i]=i;
		dis.rank[i]=0;
	}
}
void display()
{
	int i;
	printf("\nParent");
	for(i=0;i<dis.n;i++)
	{
		printf("%d\t",dis.parent[i]);
	}
	printf("\nRank");
	for(i=0;i<dis.n;i++)
	{
		printf("%d\t",dis.rank[i]);
	}
```

```
}

int find(int x)
{
	if(dis.parent[x]!=x)
	{
		dis.parent[x]=find(dis.parent[x]);
	}
	return dis.parent[x];
}

void Union(int x,int y)
{
	int setx=find(x);
	int sety=find(y);
	if(setx==sety)
		return;
	if(dis.rank[setx]>dis.rank[sety])
	{
		dis.parent[sety]=setx;
		dis.rank[sety]=-1;
```

```
        }
        else if(dis.rank[setx]<dis.rank[sety])
        {
                dis.parent[setx]=sety;
                dis.rank[setx]=-1;
        }
        else
        {
                dis.parent[sety]=setx;
                dis.rank[setx]+=1;
                dis.rank[sety]=-1;
        }
}
void main()
{
        int choice,x,y;
        while(1)
        {
                printf("\nDisjoint Set Operations");
                printf("\n------------------------");
                printf("\n1.Make Sets");
```

```
printf("\n2.Find");
printf("\n3.Union");
printf("\n4.Display");
printf("\n5.Exit");
printf("\nEnter your choices:");
scanf("%d",&choice);
switch(choice)
{
        case 1:printf("\nEnter no of elements in a set:");
                scanf("%d",&dis.n);
                makeset();
                break;
        case 2:printf("\nEnter two elements to find they are connected:");
                scanf("%d%d",&x,&y);
                if(find(x)==find(y))
                        printf("\n%d and %d are connected to a single leader
                        %d",x,y,find(x));
                else
                        printf("\nDisconnected components");break;
        case 3: printf("\nEnter two elements to perform union:");
                scanf("%d%d",&x,&y);
```

```
                    Union(x,y); break;
                case 4:display();
                    break;
                case 5:exit(1);
            }
        }
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:

```
Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice :1

Enter number of elements in a set:8

Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice :3

Enter two elements to perform union: 1 2
```

```
Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice :3

Enter two elements to perform union:1 3

Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice :3
```

```
Enter two elements to perform union:5 6

Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice :4

Parent0 1       1       1       4       5       5       7
Rank0   1       -1      -1      0       1       -1      0
Disjoint Set Operations
-----------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choice : 2
```

```
Enter two elements to find they are connected:5 6

5 and 6 are connected to a single leader 5
Disjoint Set Operations
-------------------------
1.Make Sets
2.Find
3.Union
4.Display
5.Exit
Enter your choices:5
```

Experiment No: 7

Date: 10-10-2025

BINARY SEARCH TREES

AIM: To perform stack binary search tree operations

ALGORITHM:

INSERTION

STEP 1: First, create a new node using the `create_node` function, passing the `data` value as an argument.

STEP 2: Check if the `root` of the BST is `NULL`. If the root is `NULL`, it means the BST is empty. In this case, set the new node as the root node and print a message indicating that the data was inserted in the root node.

STEP 3: If the root is not `NULL`, then initialize two pointers: `temp` and `prev`. `temp` is used to traverse the BST, and `prev` keeps track of the previous node in the traversal path.

STEP 4: Enter a loop to traverse the BST. This loop continues until `temp` becomes `NULL`.

STEP 5: Inside the loop, update `prev` to point to the current node `temp`.

STEP 6: Compare the `data` value with the `data` in the current `temp` node. If `data` is greater than the current node's data, move to the right child (i.e., `temp = temp->right`). Otherwise, move to the left child (i.e., `temp = temp->left`).

STEP 7: Continue this process of moving down the tree until finding a `temp` node that is `NULL`.

STEP 8: Check whether the `data` is greater than the `data` in the `prev` node or not. If it is greater, set the right child of `prev` to the new node. Otherwise, set the left child of `prev` to the new node.

STEP 9: Print a message indicating that the data was successfully inserted into the BST.

DELETION

STEP 1: Check if the `root` of the BST is `NULL`. If the root is `NULL`, it means the tree is empty, so return `NULL` to indicate that there's nothing to delete.

STEP 2: If the `key` is less than the `data` in the current `root` node, it means the node to be deleted is in the left subtree. Recursively call the `delete` function with the left child of the current `root`, and update the left child with the result.

STEP 3: If the `key` is greater than the `data` in the current `root` node, it means the node to be deleted is in the right subtree. Recursively call the `delete` function with the right child of the current `root`, and update the right child with the result.

STEP 4: If the `key` matches the `data` in the current `root` node, found the node to be deleted.

STEP 4. a: Check if the `root` has no left child. If it doesn't, store the right child of the `root` in a temporary variable `temp`.

STEP 4.b: Check if the `root` has no right child. If it doesn't, store the left child of the `root` in a temporary variable `temp`.

STEP 4.c: If the `root` has both left and right children, need to find the smallest node in the right subtree of the `root` (the node with the minimum value in the right subtree). By calling the `smallest_node` function, which should return the smallest node in the tree with the provided node as its root. Store the `data` of this smallest node in a temporary variable `temp`, and then update the `data` of the `root` with this `temp` value.

STEP 4.d: Recursively call the `delete` function to remove the node with the `key` from the right subtree of the current `root` with the `temp` value as the `key`.

STEP 5: Return the modified `root` after deletion.

SEARCHING

STEP 1. Initialize a temporary node pointer `temp` to the root of the BST.

STEP 2: Enter a loop that continues as long as `temp` is not `NULL`.

STEP 3: Inside the loop, compare the `key` with the `data` in the current node `temp`.

STEP 4: If the `key` matches the `data` in the current node, return `1` to indicate that the key is found in the BST.

STEP 5: If the `key` is greater than the `data` in the current node, update `temp` to point to its right child, as the key might be in the right subtree.

STEP 6: If the `key` is less than the `data` in the current node, update `temp` to point to its left child, as the key might be in the left subtree.

STEP 7: If the loop finishes and the `key` is not found in the BST, return `0` to indicate that the key is not present in the BST.

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <conio.h>

// structure of a node

struct node

{

   int data;

   struct node *left;

   struct node *right;

};

      typedef struct node node;

// globally initialized root pointer

struct node *root = NULL;

// function prototyping
```

```
struct node *create_node(int);
void insert(int);
struct node *delete (struct node *, int);
int search(int);
void printtree(struct node*,int);
void inorder(struct node *);
void postorder(struct node *);
void preorder(struct node *);
struct node *smallest_node(struct node *);
struct node *largest_node(struct node *);
int get_data();
int main()
{
   int userChoice;
   int userActive = 'Y';
   int data;
   struct node* result = NULL;
   while (userActive == 'Y' || userActive == 'y')
   {
         printf("\n\n------- Operations of Binary Search Tree ------\n");
         printf("\n1. Insertion");
```

```
printf("\n2. Deletion");

printf("\n3. Searching");

printf("\n4. Get Largest  Data");

printf("\n5. Get Smallest Data");

printf("\n6. Printing the Tree");

printf("\n\n-- Traversals --");

printf("\n7. Preorder ");

printf("\n8. Inorder ");

printf("\n9. Postorder ");

printf("\n10.Exit");

printf("\n\nEnter Your Choice: ");

scanf("%d", &userChoice);

printf("\n");

switch(userChoice)

{

   case 1:
         data = get_data();
         insert(data);
         break;

   case 2:
         data = get_data();
         root = delete(root, data);
         printf("\n %d deleted",data);
```

```
        break;
    case 3:
        data = get_data();
        if (search(data) == 1)
        {
           printf("\nData was found!\n");
        }
        else
        {
           printf("\nData not found!\n");
        }
        break;
    case 4:
        result = largest_node(root);

        if (result != NULL)
        {
           printf("\nLargest Data: %d\n", result->data);
        }
        break;


    case 5:
        result = smallest_node(root);
        if (result != NULL)
        {
           printf("\nSmallest Data: %d\n", result->data);
        }
        break;
    case 6:
        printf("\nBinary Search Tree\n");
        printtree(root,1);
        break;
```

```
            case 7:
                printf("\nPreorder Traversal\n");
                preorder(root);
                break;
            case 8:     printf("\nInorder Traversal\n");
                        inorder(root);
                        break;
            case 9:     printf("\nPostorder Traversal\n");
                        postorder(root);
                        break;
            case 10:    exit(0);
                        break;
            default:
                printf("\n\tInvalid Choice\n");
                break;
        }
        printf("\n__________\nDo you want to continue?(y/n) ");
        fflush(stdin);
        scanf(" %c", &userActive);
    }
    return 0;
}
// creates a new node
struct node *create_node(int data)
{
```

```
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    if (new_node == NULL)
    {
        printf("\nMemory for new node can't be allocated");
        return NULL;
    }
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}
// inserts the data in the BST
void insert(int data)
{
 struct node *temp,*prev;
    struct node *new_node = create_node(data);
    if (new_node != NULL)
    {
        // if the root is empty then make a new node as the root node
        if (root == NULL)
        {
```

```
    root = new_node;
    printf("\nData %d was inserted in Node\n", data);
    return;
}
temp = root;
prev = NULL;
// traverse through the BST to get the correct position for insertion
while (temp != NULL)
{
    prev = temp;
    if (data > temp->data)
    {
        temp = temp->right;
    }
    else
    {
        temp = temp->left;
    }
}
// found the last node where the new node should insert
if (data > prev->data)
```

```
        {
            prev->right = new_node;
        }
        else
        {
            prev->left = new_node;
        }
        printf("\nData %d was inserted in Node\n", data);
    }
}
// deletes the given key node from the BST
struct node *delete (struct node *root, int key)
{struct node *temp;
    if (root == NULL)
    {
        return root;
    }
    if (key < root->data)
    {
        root->left = delete (root->left, key);
    }
```

```
else if (key > root->data)
{
    root->right = delete (root->right, key);
}
else
{
    if (root->left == NULL)
    {
       struct node *temp = root->right;
       free(root);
       return temp;
    }
    else if (root->right == NULL)
    {
       struct node *temp = root->left;
       free(root);
       return temp;
    }
    temp = smallest_node(root->right);
    root->data = temp->data;
    root->right = delete (root->right, temp->data);
}
```

```
    return root;
}
// search the given key node in BST
int search(int key)
{
    struct node *temp = root;
    while (temp != NULL)
    {
        if (key == temp->data)
        {
            return 1;
        }
        else if (key > temp->data)
        {
            temp = temp->right;
        }
        else
        {
            temp = temp->left;
        }
    }
    return 0;
}
```

```
// finds the node with the smallest value in BST
struct node *smallest_node(struct node *root)
{
    struct node *curr = root;
    while (curr != NULL && curr->left != NULL)
   {
            curr = curr->left;
    }
    return curr;
}
// finds the node with the largest value in BST
struct node *largest_node(struct node *root)
{
    struct node *curr = root;
    while (curr != NULL && curr->right != NULL)
    {
            curr = curr->right;
    }
    return curr;
}
void printtree(struct node *t, int level) { int i;
```

```
    if (t != NULL) {
        printtree(t->right, level+1); // Print the left subtree first
        for (i = 0; i<level; i++) {
        printf("   "); // Print spaces for indentation
       }
       printf("%d\n", t->data); // Print the current node's data
        printtree(t->left, level+1); // Then print the right subtree
    }
}
// inorder traversal of the BST
void inorder(struct node *root)
{
    if (root == NULL)
    {
        return;
    }
    inorder(root->left);
    printf("%d-> ",  root->data);
    inorder(root->right);
}
// preorder traversal of the BST
void preorder(struct node *root)
{
    if (root == NULL)
    {
        return;
```

```
    }
    printf("%d-> ", root->data);
    preorder(root->left);

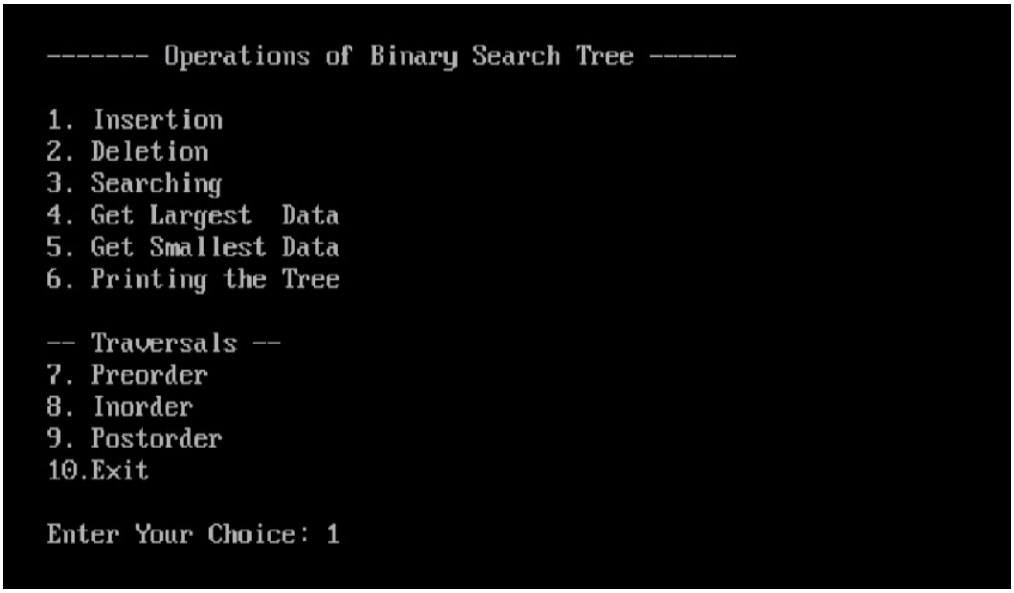
    preorder(root->right);

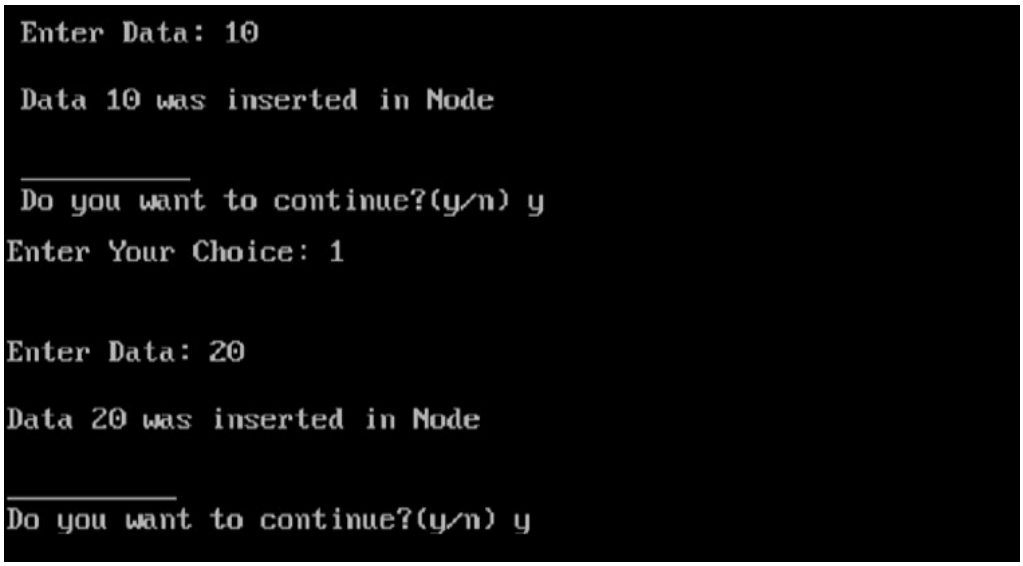
}
// postorder travsersal of the BST
void postorder(struct node *root)
{
    if (root == NULL)
    {
          return;
    }
    postorder(root->left);
    postorder(root->right);
    printf("%d-> ", root->data);
}
// getting data from the user
int get_data()
{
    int data;
    printf("\nEnter Data: ");
    scanf("%d", &data);
    return data;

}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


------- Operations of Binary Search Tree ------
1. Insertion
2. Deletion
3. Searching
4. Get Largest Data
5. Get Smallest Data
6. Printing the Tree
-- Traversals --
7. Preorder
8. Inorder
9. Postorder
10.Exit
Enter Your Choice: 1


Enter Data: 10
Data 10 was inserted in Node
Do you want to continue?(y/n) y
Enter Your Choice: 1
Enter Data: 20
Data 20 was inserted in Node
Do you want to continue?(y/n) y

```
Enter Your Choice: 1

Enter Data: 3

Data 3 was inserted in Node

__________
Do you want to continue?(y/n) y

Enter Your Choice: 1

Enter Data: 44

Data 44 was inserted in Node

__________
Do you want to continue?(y/n) y
```

```
  Enter Your Choice: 1

  Enter Data: 5

  Data 5 was inserted in Node

__________
Do you want to continue?(y/n) y
 Enter Your Choice: 6

   Binary Search Tree
         44
       20
     10
         5
       3

 __________
 Do you want to continue?(y/n) y
```

```
 Enter Your Choice: 3

 Enter Data: 10

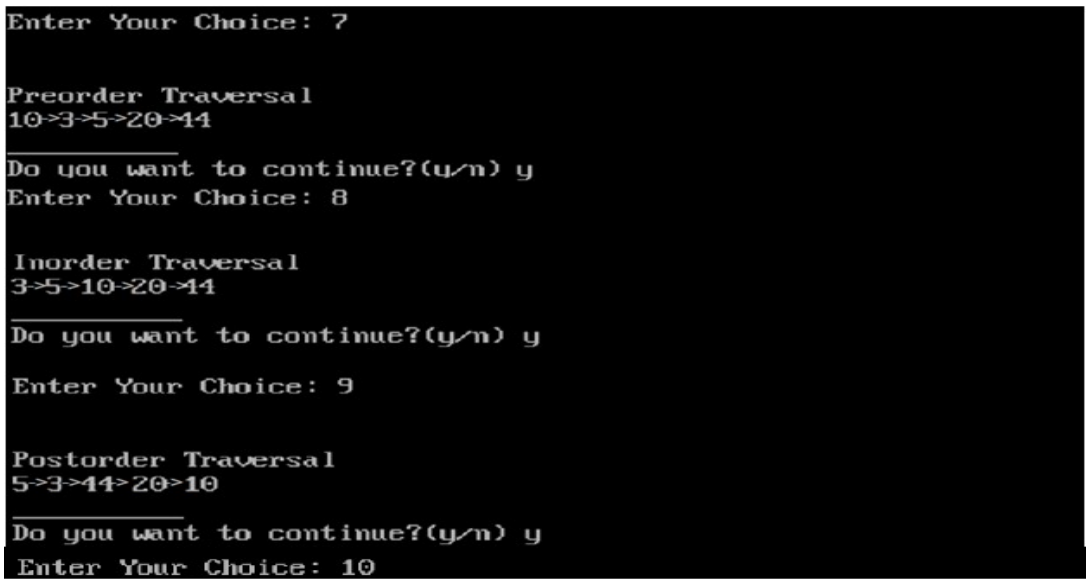
 Data was found!

 _________
 Do you want to continue?(y/n) y
 Enter Your Choice: 4

 Largest Data: 44
_________
Do you want to continue?(y/n) y
 Enter Your Choice: 5

 Smallest Data: 3

 _________
 Do you want to continue?(y/n) y
```


Enter Your Choice: 7
Preorder Traversal
10->3->5->20->44
Do you want to continue?(y/n) y
Enter Your Choice: 8
Inorder Traversal
3->5->10->20->44
Do you want to continue?(y/n) y
Enter Your Choice: 9
Postorder Traversal
5->3->44->20->10
Do you want to continue?(y/n) y
Enter Your Choice: 10

Experiment No: 8

Date: 23-10-2025

RED BLACK TREE

AIM: To implement a Red Black Tree and insert a node in it.

ALGORITHM:

Insert()

STEP 1 : Check whether tree is Empty.

STEP 2 : If tree is Empty then insert the newNode as Root node with color Black and exit from the operation

STEP 3 : If tree is not Empty then insert the newNode as leaf node with color Red.

STEP 4 : If the parent of newNode is Black then exit from the operation.

STEP 5 : If the parent of newNode is Red then check the color of parent node's sibling of newNode.

STEP 6 : If it is colored Black or NULL then make suitable Rotation and Recolor it.

STEP 7 : If it is colored Red then perform Recolor. Repeat the same until tree becomes Red Black Tree.

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

#define RED 'R'

#define BLACK 'B'

struct Node

{

   int data;

   char color;

   struct Node *left, *right, *parent;

};
```

```
struct Node *root = NULL;

// Function prototypes

void leftRotate(struct Node *x);

void rightRotate(struct Node *y);

void color(struct Node *z);

void insert(int val);

void printTree(struct Node *root, int level, int space);

// Function to perform a left rotation

void leftRotate(struct Node *x)

{

   struct Node *y = x->right;

   x->right = y->left;

   if (y->left != NULL)

      y->left->parent = x;

   y->parent = x->parent;

   if (x->parent == NULL)

      root = y;

   else if (x == x->parent->left)

      x->parent->left = y;

   else

      x->parent->right = y;
```

```
    y->left = x;
    x->parent = y;
}
// Function to perform a right rotation
void rightRotate(struct Node *y)
{
    struct Node *x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;
    x->parent = y->parent;
    if (y->parent == NULL)
        root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;
    x->right = y;
    y->parent = x;
```

```
}
// Function to adjust the color of the nodes after insertion
void color(struct Node *z)
{
   while (z->parent != NULL && z->parent->color == RED)
   {
      if (z->parent == z->parent->parent->left)
      {
         struct Node *y = z->parent->parent->right;

         if (y != NULL && y->color == RED)
         {
            z->parent->color = BLACK;
            y->color = BLACK;
            z->parent->parent->color = RED;
            z = z->parent->parent;
         }
         else
         {
            if (z == z->parent->right)
            {
```

```
            z = z->parent;
            leftRotate(z);
        }
        z->parent->color = BLACK;
        z->parent->parent->color = RED;
        rightRotate(z->parent->parent);
    }
}
else
{
    struct Node *y = z->parent->parent->left;

    if (y != NULL && y->color == RED)
    {
        z->parent->color = BLACK;
        y->color = BLACK;
        z->parent->parent->color = RED;
        z = z->parent->parent;
    }
    else
    {
```

```
            if (z == z->parent->left)
            {
                z = z->parent;
                rightRotate(z);
            }
            z->parent->color = BLACK;
            z->parent->parent->color = RED;
            leftRotate(z->parent->parent);
        }
    }
    root->color = BLACK;
}
// Function to insert a node into the Red-Black Tree
void insert(int val)
{
    struct Node *z = (struct Node *)malloc(sizeof(struct Node));
     struct Node *y = NULL;
    struct Node *x = root;
    z->data = val;
    z->left = z->right = z->parent = NULL;
```

```
    z->color = RED;
    while (x != NULL)
    {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }
    z->parent = y;
    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;
    color(z);
}
// Function to print the Red-Black Tree in a hierarchical structure
void printTree(struct Node *root, int level, int space)
{
    int i;
```

```
    if (root == NULL)
        return;
    space += 5;
    printTree(root->right, level + 1, space);
    printf("\n");
    for (i = 5; i < space; i++)
        printf(" ");
    printf("%d%c\n", root->data, root->color);
    printTree(root->left, level + 1, space);
}
void main()
{
    int ch, val;
   clracr();
    while (1)
    {
        printf(“\nRed Blcak Tree Operation”);
        printf("\n1.Insert\n2.Display\n3.Exit\nEnter choice : ");
        scanf("%d", &ch);
        switch (ch)
        {
        case 1:
            printf("Enter element : ");
```

```
            scanf("%d", &val);
            insert(val);
            break;
        case 2:
            printf("Red-Black Tree Structure:\n");
            printf("------------------------\n");
            printTree(root, 1, 0);
            printf("------------------------\n");
            break;
        case 3:
            exit(0);
        }
    }
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:

```
 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 21

 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 11

 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 35
```

```
 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 51

 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 60

 Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 1
Enter element : 20
```


Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 2
Red-Black Tree Structure:
60R
51B
35R
21B
20R
11B
Red Black Tree Operations
1.Insert
2.Display
3.Exit
Enter choice : 3

Experiment No: 9

Date: 30-10-2025

GRAPH TRAVERSAL TECHNIQUES :BFS, DFS, TOPOLOGICAL SORTING

AIM: To perform BFS , DFS , Topological sorting.

ALGORITHM:

STEP 1 : Define a Queue of size total number of vertices in the graph.

STEP 2 : Declare three choices BFS, DFS, Topological sorting.

STEP 3 : If the option selected is BFS

STEP 3.1 : Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.

STEP 3.2 : Visit all the non-visited adjacent vertices of the vertex which is at the front of the Queue and insert them into the Queue.

STEP 3.3 : When there is no new vertex to be visited from the vertex which is at the front of the Queue then delete that vertex.

STEP 3.4 : Repeat STEPs 3 and 4 until queue becomes empty.

STEP 3.5 : When queue becomes empty, then produce the final spanning tree.

STEP 4 : If the option selected is DFS

STEP 4.1 : Define a Stack of size total number of vertices in the graph.

STEP 4.2 : Select any vertex as starting point for traversal. Visit that vertex and push it on to the Stack.

STEP 4.3 : Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack

STEP 4.4 : Repeat STEP 4.3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

STEP 4.5 : When there is no new vertex use backtracking and pop one vertex the stack.

STEP 4.6 : Repeat STEPs 4.3, 4.4 and 4.5 until stack becomes Empty.

STEP 4.7 : When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph.

STEP 5 : If the option selected is Topological Sorting

STEP 5.1 : Define a Stack of size total number of vertices in the graph.

STEP 5.2 : Select the vertex which has the least degree. Remove the vertex and its associated vertex. Push it on to the Stack.

STEP 5.3 : Now update the indegree of other vertices.

SOURCE CODE:

```
#include<stdio.h>

#include<conio.h>

#define MAX 100

int a[20][20],q[20],visited[20],n,i,j,f=0,r=-1,indegree[10];

int adj[MAX][MAX],reach[20];

int queue[MAX], front = -1,rear = -1;

void main()

{

 int ch,v,j,G[MAX][MAX],count=0;


 do

 {

        Printf(“\nGraph Traversal”);

        printf("\n1 BFS\n2 DFS\n3 Topological Sort\n4 Exit \nEnter Choice:");

  scanf("%d",&ch);

  switch(ch)

  {

    case 1:
```

```
        printf("Enter the number of vertices:");
        scanf("%d",&n);
        for(i=1;i<=n;i++)
        {
          q[i]=0;
          visited[i]=0;
        }
        printf("\nEnter graph data in matrix form:\n");
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        scanf("%d",&a[i][j]);
        printf("\nEnter the starting vertex:");
        scanf("%d",&v);
        visited[v]=1;
        printf("%d\t",v);
        bfs(v);
        break;
case 2:
        printf("\n Enter the number of vertices:");
        scanf("%d",&n);
        for (i=1;i<=n;i++)
```

```
{
reach[i]=0;
for (j=1;j<=n;j++)
a[i][j]=0;
        }
        printf("\n Enter the adjacency matrix:\n");
        for (i=1;i<=n;i++)
for (j=1;j<=n;j++)
scanf("%d",&a[i][j]);
        dfs(1);
        printf("\n");
        for (i=1;i<=n;i++)
        {
if(reach[i])
count++;
        }
        if(count==n)
printf("\n Graph is connected");
        else
printf("\n Graph is not connected");
        break;
```

```
        case 3:
                printf("\n Enter the number of vertices:");
                scanf("%d",&n);
                printf("Enter the adjacency matrix:\n ");
                for(i=0;i<n;i++)
                {
                for(j=0;j<n;j++)
                scanf("%d",&a[i][j]);
                }
                topology();

                break;
            case 4:
                exit(0);
            default:
                printf("Invalid input");
            }
    }while(ch!=5);
}
int bfs(int v)
{
```

```
 for(i=1;i<=n;i++)
   if(a[v][i] && !visited[i])
     q[++r]=i;
   if(f<=r)
   {
   visited[q[f]]=1;
   printf("%d\t",q[f]);
   bfs(q[f++]);
   }
 return 0;
}
int dfs(int v)
{
 int i;
 reach[v]=1;
 for (i=1;i<=n;i++)
  if(a[v][i] && !reach[i])
  {
     printf("\n %d->%d",v,i);
     dfs(i);
}
```

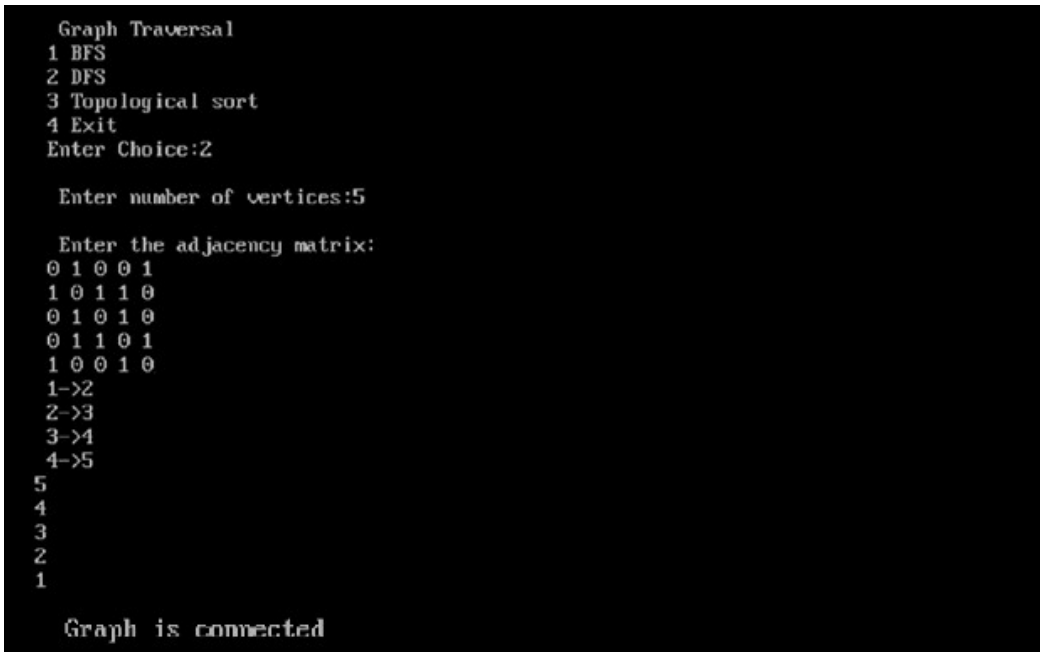
```
    printf("\n%d",v);
return 0;
}
void find_indegree()
{
 int i,j,sum;
 for(j=0;j<n;j++)
 {
  sum=0;
  for(i=0;i<n;i++)
   sum+=a[i][j];
 indegree[j]=sum;
 }
}
int topology()
{
 int i,u,v,t[10],s[10],top=-1,k=0;
 delay(1000);
 find_indegree();
 for(i=0;i<n;i++)
 {
  if(indegree[i]==0)s[++top]=i;
```

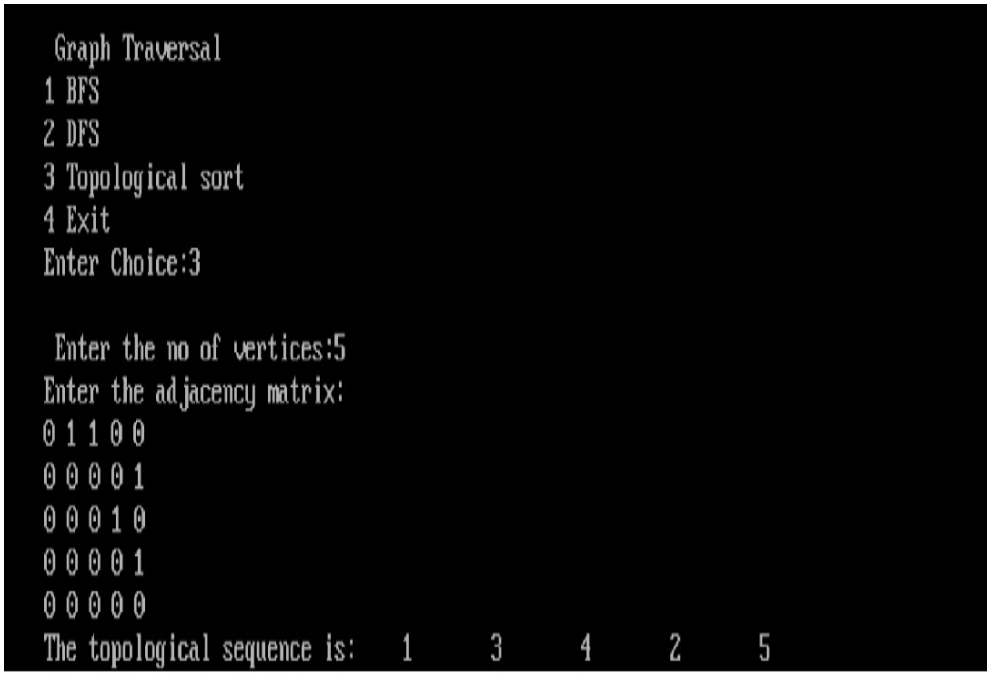
```
}
while(top!=-1)
{
 u=s[top--];
 t[k++]=u;
 for(v=0;v<n;v++)
 {
  if(a[u][v]==1)
  {
   indegree[v]--;
   if(indegree[v]==0)s[++top]=v;
  }
 }
}
 printf("The Topological Sequence is:");
 for(i=0;i<n;i++)
 printf("\t%d",t[i]+1);
 return 0;
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Graph Traversal
1 BFS
2 DFS
3 Topological sort
4 Exit
Enter Choice:1
Enter the no of vertices:5
Enter graph data in matrix form:
0 0 1 1 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 1
0 1 0 0 0
Enter the starting vertex:1
1 3 4 5 2


Graph Traversal
1 BFS
2 DFS
3 Topological sort
4 Exit
Enter Choice:2
Enter number of vertices:5
Enter the adjacency matrix:
0 1 0 0 1
1 0 1 1 0
0 1 0 1 0
0 1 1 0 1
1 0 0 1 0
1->2
2->3
3->1
4->5
5
4
3
2
1
Graph is connected


Graph Traversal
1 BFS
2 DFS
3 Topological sort
4 Exit
Enter Choice:3
Enter the no of vertices:5
Enter the adjacency matrix:
0 1 1 0 0
0 0 0 0 1
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
The topological sequence is: 1 3 4 2 5

Experiment No: 10

Date: 06-11-2025

PRIM'S ALGORITHM

AIM: To perform minimum cost spanning tree using prim's algorithm

ALGORITHM:

STEP 1 : Select a starting vertex

STEP 2 : Repeat Steps 3 and 4 until there are fringe vertices

STEP 3 : Select an edge e connecting the tree vertex and fringe vertex that has minimum weight.

STEP 4 : Add the selected edge and the vertex to the minimum spanning tree T.
[END OF LOOP]

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
int a,b,u,v,n,i,j,nedge=1;
int visited[10]={0},min,mincost=0,cost[10][10];
void main()
{

    printf("\nMinimum Cost Spanning Tree\n");
    printf("Implementation of Prim's Algorithm");
    printf("\nEnter the number of nodes:");
    scanf("%d",&n);
    printf("\nEnter the adjacency matrix:\n");
    for(i=1;i<=n;i++)
        {for(j=1;j<=n;j++)
        {
            scanf("%d",&cost[i][j]);
            if(cost[i][j]==0)
            cost[i][j]=999;
```

```
            }
        visited[1]=1;
        printf("\n");
        }
        while(nedge<n)
        {
            for(i=1,min=999;i<=n;i++)
                for(j=1;j<=n;j++)
                    if(cost[i][j]<min)
            if(visited[i]!=0)
            {
                min=cost[i][j];
                a=u=i;
                b=v=j;
            }
            if(visited[u]==0||visited[v]==0)
            {

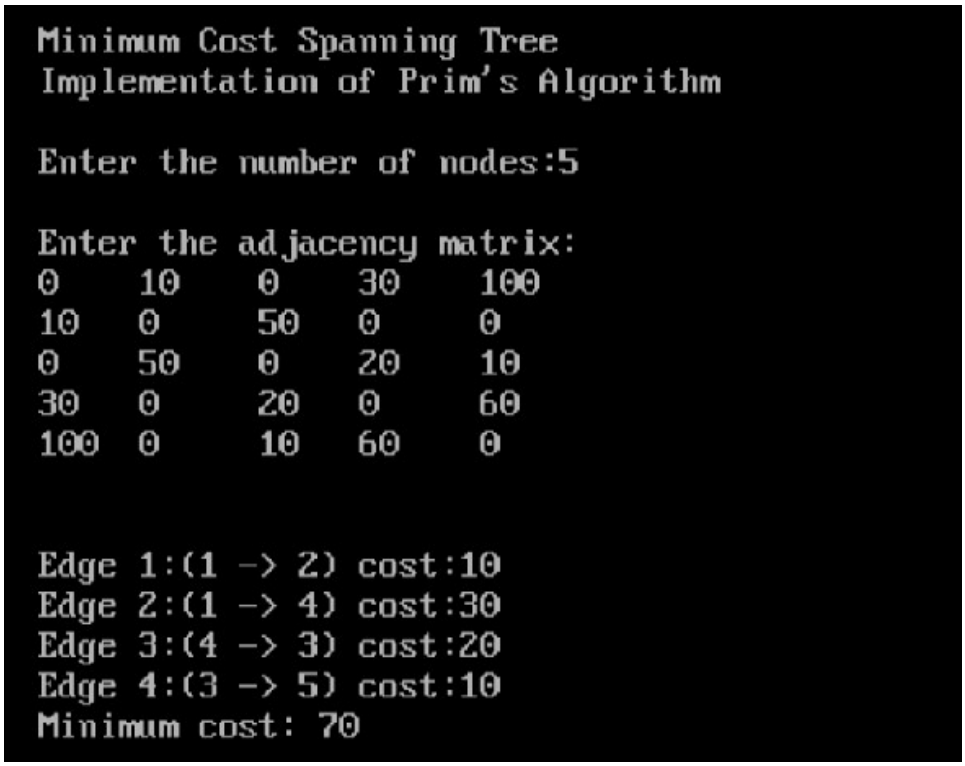
                printf("\nEdge %d:(%d  -> %d) cost:%d",nedge++,a,b,min);
                mincost+=min;
                visited[b]=1;
            }
        }
            cost[a][b]=cost[b][a]=999;

        }
        printf("\nMinimum cost: %d\n",mincost);

}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Minimum Cost Spanning Tree
Implementation of Prim's Algorithm
Enter the number of nodes:5
Enter the adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Edge 1:(1 -> 2) cost:10
Edge 2:(1 -> 4) cost:30
Edge 3:(4 -> 3) cost:20
Edge 4:(3 -> 5) cost:10
Minimum cost: 70

Experiment No: 11

Date:13-11-2025

KRUSKAL'S ALGORITHM

AIM: To perform minimum cost spanning tree using Kruskal's algorithm

ALGORITHM:

STEP 1 : Create a forest in such a way that each graph is a separate tree.

STEP 2 : Create a priority queue Q that contains all the edges of the graph.

STEP 3 : Repeat STEP 4 and 5 while Q is NOT EMPTY

STEP 4 : Remove an edge from Q

STEP 5 : If the edge obtained in STEP 4 connects two different trees, then Add it to the forest (for combining two trees into one tree).

STEP 6 : Else Discard the edge

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
int i,j ,a,b,u,v,n,nedge=1;
int min,mincost=0,cost[9][9],parent[9];

int find(int);

int uni(int,int);

void main()
{

printf("Minimum Cost Spanning Tree\n");
printf("\nImplementation of Kruskal's Algorithm\n");

printf("\nEnter the number of vertices:");

scanf("%d",&n);

printf("\nEnter the cost using adjacency matrix:\n");

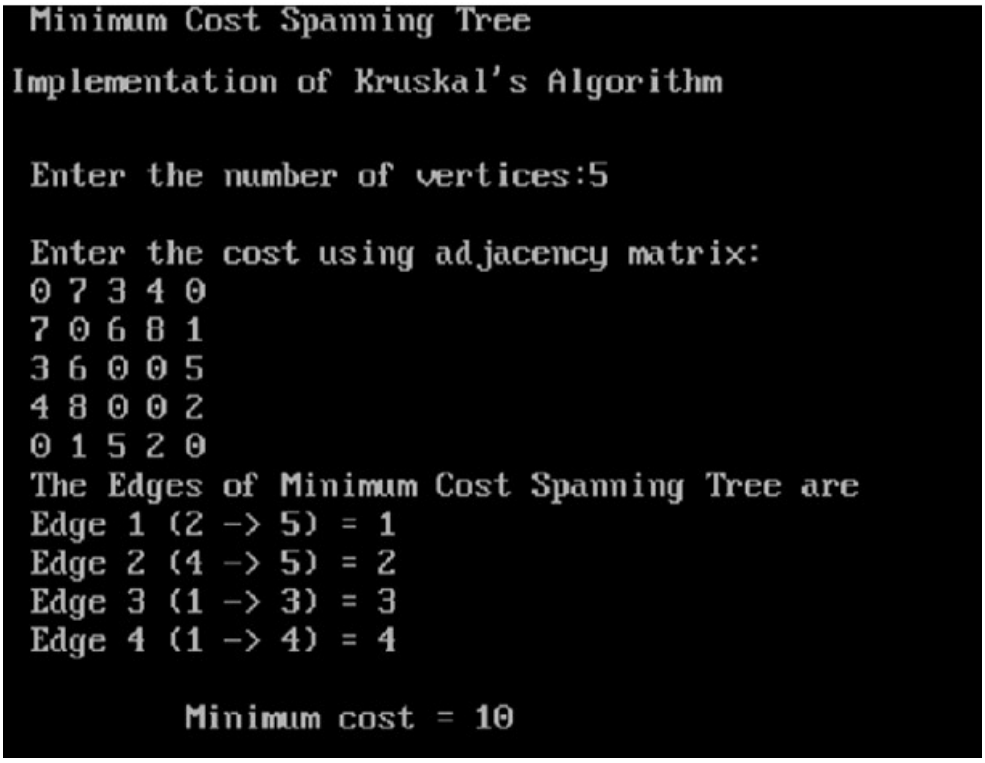
for(i=1;i<=n;i++)
```

```
        {
                for(j=1;j<=n;j++)
                {
                        scanf("%d",&cost[i][j]);
                        if(cost[i][j]==0)
                                cost[i][j]=999;
                }
        }
printf("The Edges of Minimum Cost Spanning Tree are\n");
while(nedge < n)
        {
                for(i=1,min=999;i<=n;i++)
                {
                        for(j=1;j <= n;j++)
                        {
                                if(cost[i][j] < min)
                                {
                                        min=cost[i][j];
                                        a=u=i;
                                        b=v=j;
                                }
                        }
                }
                u=find(u);
                v=find(v);
```

```
                if(uni(u,v))
                {
                        printf("Edge %d (%d -> %d) = %d\n",nedge++,a,b,min);
                        mincost +=min;
                }
        cost[a][b]=cost[b][a]=999;
        }
printf("\n\tMinimum cost = %d\n",mincost);
}
int find(int i)
{
        while(parent[i])
                i=parent[i];
        return i;
}
int uni(int i,int j)
{
        if(i!=j)
        {
                parent[j]=i;
                return 1;
        }
        return 0;
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Minimum Cost Spanning Tree
Implementation of Kruskal's Algorithm
Enter the number of vertices:5
Enter the cost using adjacency matrix:
0 7 3 4 0
7 0 6 8 1
3 6 0 0 5
4 8 0 0 2
0 1 5 2 0
The Edges of Minimum Cost Spanning Tree are
Edge 1 (2 -> 5) = 1
Edge 2 (4 -> 5) = 2
Edge 3 (1 -> 3) = 3
Edge 4 (1 -> 4) = 4
Minimum cost = 10

Experiment No: 12

Date: 20-11-2025

SINGLE SOURCE SHORTEST PATH : DIJKSTRA'S ALGORITHM

AIM: To find the single source shortest path algorithm.

ALGORITHM:

STEP 1 : Create a set shortPath to store vertices that come in the way of the shortest path tree.

STEP 2 : Initialize all distance values as INFINITE and assign distance values as 0 for source vertex so that it is picked first.

STEP 3 : Loop until all vertices of the graph are in the shortPath.

STEP 3.1 : Take a new vertex that is not visited and is nearest.

STEP 3.2 : Add this vertex to shortPath.

STEP 3.3 : For all adjacent vertices of this vertex update distances. Now check every adjacent vertex of V, if sum of distance of u and weight of edge is else then update it.

SOURCE CODE:

```
#include<stdio.h>

#include<conio.h>

#define INFINITY 9999

#define MAX 10

void dijkstra(int G[MAX][MAX],int n,int startnode);

void main()

{

int G[MAX][MAX],i,j,n,u;

printf("\nSingle Shortest Path\n");

printf("Enter the number of vertices:");
```

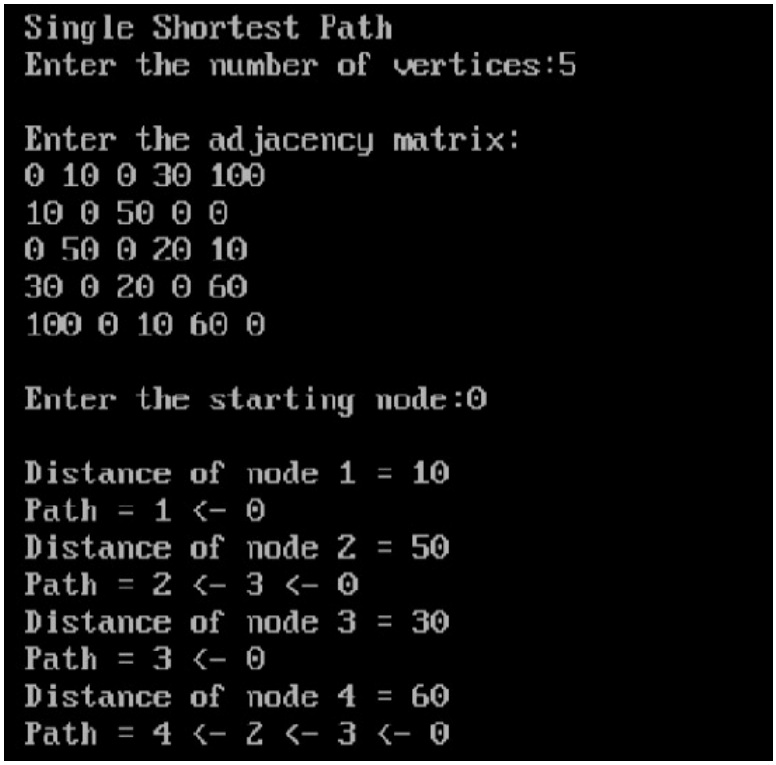
```
scanf("%d",&n);
printf("\nEnter the adjacency matrix:\n");
for(i=0;i<n;i++)
        for(j=0;j<n;j++)
                scanf("%d",&G[i][j]);
printf("\nEnter the starting node:");
scanf("%d",&u);
dijkstra(G,n,u);
}
void dijkstra(int G[MAX][MAX],int n,int startnode)
{
int cost[MAX][MAX],distance[MAX],pred[MAX];
int visited[MAX],count,mindistance,nextnode,i,j;
for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        {
                if(G[i][j]==0)
                        cost[i][j]=INFINITY;
                else
                        cost[i][j]=G[i][j];
        }
```

```
for(i=0;i<n;i++)
{
        distance[i]=cost[startnode][i];
        pred[i]=startnode;
        visited[i]=0;
}
distance[startnode]=0;
visited[startnode]=1;count=1;
while(count<n-1)
{
        mindistance=INFINITY;
        for(i=0;i<n;i++)
                if(distance[i]<mindistance&&!visited[i])
                {
                mindistance=distance[i];
                nextnode=i;
                }
visited[nextnode]=1;
for(i=0;i<n;i++)
        if(!visited[i])
                if(mindistance+cost[nextnode][i]<distance[i])
                {
```

```
                    distance[i]=mindistance+cost[nextnode][i];
                    pred[i]=nextnode;
                }
count++;
}
for(i=0;i<n;i++)
        if(i!=startnode)
        {
        printf("\nDistance of node %d = %d",i,distance[i]);
        printf("\nPath = %d",i);
        j=i;
        do
        {
                j=pred[j];
                printf(" <- %d",j);
        }while(j!=startnode);
        }
}
```

RESULT: Output is obtained and the result is verified.

OUTPUT:


Single Shortest Path
Enter the number of vertices:5
Enter the adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter the starting node:0
Distance of node 1 = 10
Path = 1 <- 0
Distance of node 2 = 50
Path = 2 <- 3 <- 0
Distance of node 3 = 30
Path = 3 <- 0
Distance of node 4 = 60
Path = 4 <- 2 <- 3 <- 0